

Waipapa Taumata Rau
University of Auckland

ENGINEERING AND DESIGN

DEPARTMENT OF ELECTRICAL, COMPUTER, AND SOFTWARE ENGINEERRING

2026 Part IV Research Project

Literature Review and Statement of Research Intent

TITLE HERE

Prepared by: Ming Ray Goy (mgoy303)

Project number: 43
Project partner: Ky Khon Quach
Supervisors: Dr. Valerio Terragni

Declaration of Originality
This report is my own unaided work and was not copied from anyone or anywhere nor written in collaboration with any other person.

1 Literature Review

1.1 Testing

Given a computer program, there is a need to determine its capability to function exactly as it has been designed to do. This is relates to the concept of program testing, which is used in applications and software to detect the presence of errors, and increase confidence in the program's reliability.

1.1.1 Types

Quadri and Farooq [1] discuss the different testing objectives that vary depending on the context and purpose. Factors include software type, user expectations, requirements and development methodologies. This can be seen through the different testing contexts, for example, unit testing, integration testing and system testing, that all work to validate functionalities of a program so it meets its requirements and to prevent unexpected outcomes.

There are two main types of analysis on a program, static and dynamic analysis [2]. Software testing, is an example of the latter, which relies on the execution of the program to determine correctness. A general model of this can described by Huang [3] which involves running test cases on the program with test inputs and expected outputs. This intends to improve program correctness by providing a structured measure of quality. The paper which uses flow charts and graphs to represent program state also highlights that it is impractical to test for every single path a program execution can follow, due to the combinatorial growth caused by branches, loops, etc. This reinforces the limitation of software testing above. Testing can reveal defects and errors, but cannot prove their absence. This means that the application of testing comes as a strategic decision, as it cannot always determine the root cause of failures, provide full coverage or guarantee complete program correctness.

1.1.2 Techniques

Across the software development industry, there are various different testing techniques that share the overall purpose of improving software quality, although differing in scope and methodology. From Luo [2], we see that the testing techniques can be separated into two main categories. Structural testing where the software is transparent, with the selection of test cases based on the implementation and functional testing, where the software is viewed as a "black box" with test cases selected based on requirement or design specification under test. The expected results of such testing can be called test oracles.

Some examples of different testing techniques include random testing that is structurally based with where test cases are generated with random inputs, fuzz testing [4] that is essentially a functionally (blackbox) based version of random testing, model based testing [5] where tests are generated based on a defined system model, etc which all involve execution of test cases with inputs and outputs in some way.

1.1.3 Oracle Problem

A test oracle is a mechanism that determines whether a system's observed behaviour is correct or incorrect given an input. From Barr et al.[6], several forms of oracles are discussed, including specified oracles that are based on formal specifications and direct assertions with specific inputs

giving specific outputs. However in real world scenarios, many programs and systems either cannot be defined clearly via formal specifications or they fall out of date, as stated in the study. In these situations, other oracle types, for example derived and implicit oracles, that are based on inferred information from documentation, program executions and system properties under test or manually created oracles based on human noticed properties are considered. However this is not something that can be done easily, as there might not have enough information of the system to derive oracle relations and humans oracles suffer from subjectivity, inconsistency and high cost. This gap manifests as the oracle problem in software testing, which describes the challenge of testing programs without adequate and specified test oracles. Consequently, the oracle problem is also one of the main limitations of test automation, as the ability to automate test generation via generated test oracles would allow automation of software testing itself as the study by Pastore et al. [7] here mentions and attempts to create oracles through crowdsourcing.

1.2 Metamorphic Testing

Metamorphic testing is a type of functional testing. The concept involves defining metamorphic relations (MRs) which are relations or necessary properties between at least two or more related inputs of a program and their corresponding expected outputs with relations that can help with determining the correctness of the program, even if the actual correctness of any specific input is unknown [8, 9].

Chen et al. [8] formally expresses an MR as $R(x_1,x_2...x_k...x_n, f(x_1),f(x_2),f(x_k),...f(x_n))...)$. The full sequence $<x_1,x_2...x_k...x_n>$ is known as a metamorphic group (MG) of inputs for the MR which comprises of source inputs from x_1 to x_k and follow up inputs derived from them x_k to x_n. These follow up inputs are used to test the program and are compared against the follow up outputs $f(x_k)$ to $f(x_n)$ derived from the outputs $f(x_1)$ to $f(x_k)$ of the original source inputs. This is done by observing the behavior of the program from its execution when supplied with multiple predefined inputs from MRs. [10] states that that violations of MRs are an indication that the program is faulty, however it is important to note that absence of violations does not mean complete correctness of a program, as just like other testing techniques MT does not exhaustively cover all possible inputs and resultant relations in the program.

Xu et al. [10] and Chen et al.[8] provide examples of MRs. One shared example considers the shortest-path algorithm, where for the relation for a given program P modelling a undirected graph G for vertices a and b, $G|P(a,b,G)| = G|P(b,a,G)|$ (or in words, swapping the vertices in an undirected graph will result in equal path lengths) holds true. The example illustrates that if this MR is used to test the program, the MR's satisfaction can only be determined after observing and comparing the generated output (path lengths from generated graph) from multiple source inputs (P(a,b,g) and P(b,a,g) in this case), which is much easier than determining the expected output of either input individually beforehand without execution of the system.

1.2.1 Advantages

Most studies involving metamorphic testing, [8, 9, 10], mention MT as a solution to the oracle problem, as it does not require a traditional or specified test oracle. Conceptually MT is simple, with Liu [11] showing that testers being able to learn and apply MT in different systems. It is also mentioned that after MRs are determined within a system, the implementation and integration of

the relations are straightforward with examples shown in studies on program repair and debugging, [12, 13]. With MT, the limitation to automation in the oracle problem before is also greatly reduced, with Xu et al. [10] from before researching automated generation of test cases from MRs that are derived from existing test cases in the program, potentially bringing us one step closer to full test automation of programs. It is to be noted that although MT requires additional effort to identify suitable metamorphic relations, the cost of generating follow up test cases via systematic transformations is relatively low. In context of scalability, MT becomes a valuable technique as it is not constrained by minimum test-suite size and can be applied to programs of any size and complexity.

1.2.2 Comparison to Property Based Testing

Metamorphic testing (MT) can be considered a specific type of property based testing [14], designed for situations where the exact expected output property is difficult to specify. Both MT and PBT rely on general, high-level properties that constrain the relationship between inputs and outputs rather a fixed set. However, there are important differences.

Property based testing is an approach where the expected behaviour of the system is specified through general properties rather than individual test cases [14]. Instead of checking specific input-output pairs such as in random testing, a set of defined properties and rules are used to test against the program. An example of such given property in the study is "reversing a list twice gives the original list". This allows for a wider test range that may reveal more bugs as the program is tested on more than just fixed execution paths.

PBT is more general and can express a wide range of properties—such as invariants, post-conditions, model-based checks, and other universal properties that are usually evaluated over single executions within a program. In contrast, MT focuses on metamorphic relations that link multiple executions of the program, where a source input is transformed into one or more follow-up inputs, and the corresponding outputs are checked against a known relationship. This makes MT particularly useful compared to PBT in systems where a concrete test oracle is hard or impossible to define, as it MRs are specifically designed to allow testers to measure correctness in terms of relative behaviour.

1.2.3 Challenges in MT Application

Despite MT's effectiveness in alleviating the oracle problem, most studies involving MT have noted several challenges faced that hinder its practical applicability.

One of the main challenges is the lack of awareness of the concept of MT. Chen et al. [8] emphasizes that MT is still not widely adopted into mainstream software development processes despite being a topic of growing research. Research in the concept of MT seems to have started around 1998, with the technical report of [15]. Segura et al. [16] states that research on MT is "scattered" and limited. Hoard [9] also shows through results of surveys towards open source software (OSS) that many developers have little to no experience to the MT. Besides that, as a relatively new concept in testing, availability of appropriate learning materials for MT is an issue as Chen et al. [8] also mentions that in-depth teaching is required for effective MT application in sofware testing. These reasons resulting in overall lack of awareness and proper understanding of MT is likely a main barrier towards its adoption, as developers unfamiliar with a concept will find it difficult to

integrate with existing workflows.

The importance of having proper understanding of MT leads to another main conceptual challenge of MT application, which is the difficulty in identifying suitable metamorphic relations. MRs being the fundamental part of MT requires deep knowledge of the context and domain of the program and its expected behaviour. As mentioned before and from Chen et al. [8], identification and choosing MRs is a required overhead in comparison to other testing techniques, however it is an unavoidable step in functional testing situations where we are unable to derive specified test oracles. Depending on the context, MR identification can be time-consuming and often require manual confirmation. Rehman et al. [17] discusses a good example involving MT usage for testing machine learning programs, with mentions of multiple challenges it faces in identifying MRs suitable for ML models, as it is difficult to form meaningful MRs due to the lack of explicit specifications and models that are data-driven, along with the large input space that usually comes with ML systems and require.

MR derivation for MT can be seen as a key open problem, [16], where the effectiveness of MT heavily depends on the quality of the selected relations. Currently, there seems to be no universally accepted metric, framework or tool for measuring MR quality, or on MT's theoretical foundation itself [8, 18]. As Segura et al. [16] also mentions, existing studies involving empirical methods vary significantly across studies, especially in how MR effectiveness is determined. This lack of standarised measure of MR quality results in the limitation in MT usage itself, as it directly affects MTs capability in fault detection within a program, especially with the fact that MR satisfaction does not guarantee correctness. Differentiating between strong and weak MRs is crucial to creating tests that can achieve better coverage and fault detection. A major resultant point from this is that with ambigious measures of MT effectivity, it limits the ability to compare MT as a testing technique in comparison to other studies on testing, further hindering MT's research and application in development.

Leading on from the previous point, as MT is capable of finding faults through violations, it is often used as a complementary testing technique that exists along with existing techniques. However there have not been many studies in this area, with [8] highlighting the need for better integration strategies to encourage MT adoption in development pipelines. One of such studies also with Chen et al. [19] investigate such integration with fault based testing and program analysis techniques such as metamorphic slicing.

1.2.4 Adoption and MT Usage

The adoption of metamorphic testing and its underlying concept of metamorphic relations extends beyond just software e program testing. Chen et al. [8] has cited multiple of such usages of MT being researched and aplied in areas such as bioinformatics [20], embedded systems [21], search engines [22], etc.

Within software development, there is an understandably more studies regarding its adoption, especially where traditional testing struggles to determine correctness in systems. Such studies include research by Chan et al. [23] on online testing of service oriented software using MT, as well as in software product lines, where [24, 25] investigate the use of MT in software product lines by applying MRs to feature models to automatically generate test cases for validation analysis tools.

MT has already illustrated prominent successes illustrating its effectiveness in software testing. For example applicatino of MT to test the popular C complilers, GCC and LLVM revealed over a hundred previously undetected bugs through simple metamorphic relations on program equivalence [26]. Similarly, this was also seen in the Siemens programs, where MT discovered faults missed by two decades of traditional test cases in the Siemens suite [27, 28].

Aside from that, in the world of machine learning (ML), there has also been a noticeable amount of research of using metamorphic testing on ML models and also more specifically metamorphic relations to help with the model training. From before, Rehman et al. [17]'s study on application of MT for ML states that MT has been widely accepted as an affective approach for testing both supervised and unsupervised ML systems, where MRs have been used to also assess model robustness and consistency under input transformations. Studies such as [29, 30] show research on automated testing for large language models (LLM) using metamorphic testing where the black box nature of such models would previously require a lot manual labeling and determining traditional oracles to form test cases. One specific example of MT's effectiveness in ML can be shown through the study on MT usage on driverless cars by Zhou et al. [31] that was able to detect inconsistencies in a previous perception module of Uber's autonomous system that resulted in a fatal accident.

1.3 Research Methodologies

1.3.1 Surveys, Interviews, and Questionnaires

A main method seen in multiple research papers and studies is to use surveys. This is an empirical way to gather data, which involves observing and collecting responses from individuals or groups related to the topic of research in some way, whether it be as people with experience, people matching the criteria of the study, etc. The main advantage of this methodology is the ability to understand the current state of topic in research firsthand from the responders, along with different insights that may help with improving or solving related issues discussed. Surveys can include different forms of questioning, such as structured questionnaires and in-depth interviews. For example, [32] conducted a study on software testing using both structured questionnaires and in-person tape-recorded interviews. This combination allowed for both open and closed questions providing both quantitative and qualitative feedback.

Specifically for metamorphic testing, there has not been many empirical survey-based research conducted, the only one found involving question based surveys being [9] ([16] also conducts a survey on metamorphic testing, but it uses a systematic literature review style survey on databases instead of human based questioning). Hoard Hoard [9] investigates the acceptance, adoption and perceptions of metamorphic testing of open source software (OSS) developers. This was done using a cross-sectional survey capable of capturing the responders' current thoughts and conducted using online instruments for cost effectiveness.

The survey consisted of a detailed questionnaire adapted from the Unified Theory of Acceptance and Use of Technology (UTAUT) framework that was chosen due to it being a widely used model for predicting technology use and acceptance [33]. The UTAUT framework is based of a couple key constructs: Performance expectancy (PE), Effort expectancy (EE), Behavioral intention (BI) and Frequency of Use (FoU). This questionnaire in [9] uses Likert-scale based questions where the responders are required to choose a level between 1 to 7 relating to the UTAUT constructs. These questions measure developer perceptions (PP and EE), adoption intention (BI), actual usage (FoU)

and experience level with MT.
From these questions, the survey found that many developers had little or no prior experience with MT (low FoU) confirming the previous point regarding low awareness of MT. However those who did understand MT reported higher perceived usefulness and ease of use (PP and EE), thus leading to higher adoption intentions (BI).

In this survey however, the participants were invited through email addresses associated with contributors from specifically open source Github projects on scientific software, limited to only 41 of such developers. As only developers of scientific software where MT is likely to be more relevant and used are involved, the survey overrepresents MT from the perspective of those more involved. This small and very targeted sample makes the results of the survey regarding MT adoption and perspective very likely to be inaccurate and not relatable to the general wider community of software developers. Aside from that, the lack of interviews and open based questions makes for a mostly quantitative study, which lacks the insight that detailed feedback could otherwise provide.

1.3.2 Resource Mining

Aside from human-based observations and feedback, empirical research can be conducted through "mining" resources, such as in [16] mentioned above. These sort of studies collect and analyse real data to discover patterns related to the research topic. This approach can also include both quantitative data such as from structured, measureable or numeric data and qualitative data from interpretation and context of themes, opinions and less structured data from the analysed resources. Compared to surveys, this methodology of research captures insights on real world applications of the researched topics instead of general perceptions.

In regards to software development, resource mining usually refers to analysing existing codebases to understand how and why developers have implemented certain functionalities. Consider [10, 34] that study the automated synthesis of metamorphic relations from test cases through the developed MR Scout tool. Xu et al. selected open source Java projects from Github to identify test cases that implicitly encode metamorphic relations, known as MR-encoded test cases (MTC). Over 4000 of such projects were considered where MTCs were found in 17% of the projects, with an average of around 16 MTCs per project over a total count exceeding 11000. Considering the low awareness and reported adoption of MT, this 17% is definitely significant and suggests that developers are already applying metamorphic-like reasoning implicitly. The conceptual foundations of MT have been used unconsciously without formal recognition or systematic applications. This shows a gap between the latent capability of MT and its explicit adoption, which highlights the opportunity for possible tools and frameworks to be developed that leverages this knowledge to improve overall software program testing.

Resource mining on codebases like above has its limitations. Through explicit code alone, developer intent behind the implementations are not captured well. Patterns in code could reflect unconscious practises instead of deliberate adoption of techniques, such as from the study [10] that detected that MR-like behaviour in test cases when there was no intentional use of MT. Aside from that, the selection of the dataset for the mining strongly constrains the resultant analysis to the context of the chosen sources. This is essentially the same problem that the survey [9] faces, where the specific sample chosen limits the applicability of the results of the study towards the topic of metamorphic testing in general. Research on mining software repositories (MSR) have found that

many studies do not follow a transparent selection process for repositories, which often results in studies failing to identify threats to validity of the extracted data [35]. An example of such threat if repository selection was not considered in detail would be bias caused by the inclusion of inactive, non-collaborative or non-software based repositories which are common on platforms like GitHub [36]. Furthermore, resource mining can rely on automated inference techniques based on heuristics, which may introduce inaccuracies in the data analysis.

However aside from analysing just code, files and other digital artefacts in software projects, the concept of resource mining can extend further to other online sources such as forum discussions, issue trackers, and social media, where perspectives are expressed in informal yet possibly insightful ways. For example, there has been research on mining large-scale developer discussions from platforms and related Q&A forums to understand applications of software development methodologies. An example study [37] analysed over 13000 posts using topic modelling and qualitative analysis to identify common challenges and trends in development practises through a human perspective from the popular forum platform Stack Overflow. This would not be possible by extracting and analysing source code from projects alone that only reflects what developers have implemented but not why certain approaches were used. This can possibly help solve the lack of developer intent limitation stated above.

2 Summary, Project Objectives and Scope

Metamorphic testing (MT) has shown its significant effectiveness as a software testing technique, especially in scenarios where traditional test oracles are unavailable or difficult to construct. MT has successfully identified critical faults in a variety of systems including compilers, scientific software, machine learning applications, etc which reinforces its value as a solution to the oracle problem and a great complementary approach to existing testing techniques. Despite these proven successes, MT is still underutilised in mainstream software development, due to several challenges it faces, including limited awareness of MT, absence of standardised tools and frameworks to apply and evaluate MT in development and the difficulty in identifying suitable MRs for different program contexts.

To address this gap, it is necessary to understand the current adoption and perceptions of MT by general software developers. Existing studies have attempted to do so, however with the only empirical based survey of MT being [9] that focuses on a much too narrow sample of OSS developers working on scientific programs that is too restrictive to represent the more general developer community. Aside from that, there seems to be no research conducted on understanding MT adoption through resource mining, with the studies [10, 34] from before focusing only on repository mining for Java based open source GitHub projects to detect MR related patterns instead. The scope of the project this literature review has been conducted on will thus aim to extend the studies above by involving a combination of discussed empirical methods. This will likely include conducting surveys towards a more diverse and general target audience representing the developer community and mining data from a wider range of open source software projects along with analysis of online forums and related media for more insight on MT.

Ultimately, the goal of the project is not to research and improve on MT or its underlying concept, but rather to provide a clear and comprehensive understanding of its current state of adoption in software development in general along with developers' perceptions and that may help with future development of tools, education and integration strategies of MT.

2.0.1 Research Questions and Objectives

- **RQ1**: What is the current state of awareness of MT among software developers, and what challenges and barriers do developers associate with its adoption?

  To answer this question, a broad survey of software developers will be conducted to better understand how metamorphic testing has been adopted in general and also their perspectives towards metamorphic testing. This will help identify challenges in MT adoption and the extent of influence these barriers have towards MT as a whole.

- **RQ2**: How prevalent is explicit metamorphic testing adoption across large-online resources, and what project characteristics are associated with its presence or absence?

  To investigate this, the project intends to mine and analyse a wide variety of online resources including open-source repositories and relevant online forums. This is done to detect explicit MT practises, MR-based test cases and to examine other project characteristics (e.g. programming language, project size, project type, etc) that may be relevant to MT's presence or absence.

- **RQ3**: What is the relationship between developers' perceptions of MT and the observed prevalence of MT in repositories, and what does any divergence reveal about adoption?

  To determine this, the results from the conducted surveys and findings from repository and forum analysis will be compared. This will allow for identification of possible similarities and divergences between perceived adoption and actual adoption of MT to help draw conclusions on the current state of adoption of MT and explore what these gaps indicate about MT integration into software development workflows.

3 Project Schedule and Plan

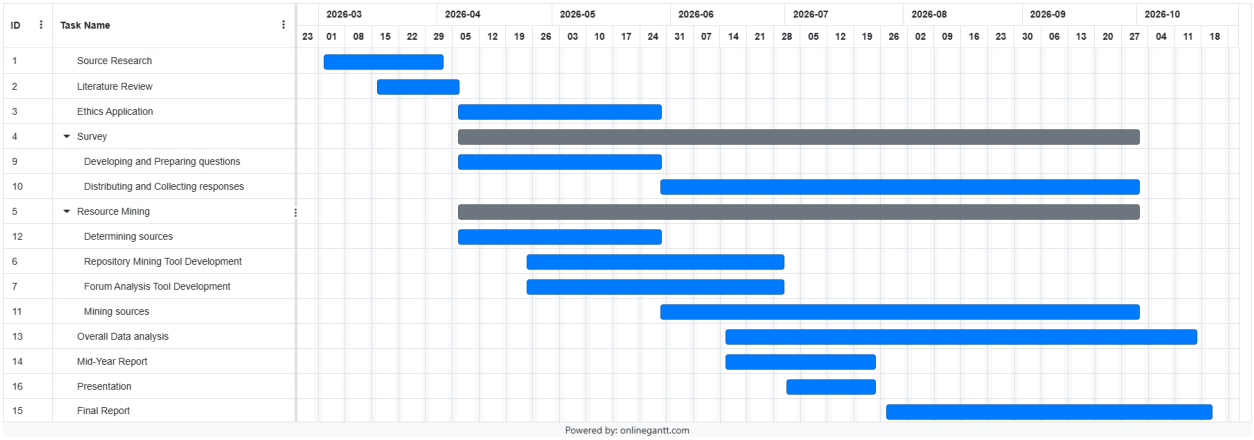


Figure 1: Project Gantt Chart

References

[1] S. Quadri and S. U. Farooq, "Software testing–goals, principles, and limitations," *International Journal of Computer Applications*, vol. 6, no. 9, p. 1, 2010.

[2] L. Luo, "Software testing techniques," *Institute for software research international Carnegie mellon university Pittsburgh, PA*, vol. 15232, no. 1-19, p. 19, 2001.

[3] J. Huang, "An approach to program testing," *ACM Computing Surveys (CSUR)*, vol. 7, no. 3, pp. 113–128, 1975.

[4] P. Godefroid, "Random testing for security: blackbox vs. whitebox fuzzing," in *Proceedings of the 2nd international workshop on Random testing: co-located with the 22nd IEEE/ACM International Conference on Automated Software Engineering (ASE 2007)*, pp. 1–1, 2007.

[5] V. Garousi, A. B. Keleş, Y. Balaman, Z. Ö. Güler, and A. Arcuri, "Model-based testing in practice: An experience report from the web applications domain," *Journal of Systems and Software*, vol. 180, p. 111032, 2021.

[6] E. T. Barr, M. Harman, P. McMinn, M. Shahbaz, and S. Yoo, "The oracle problem in software testing: A survey," *IEEE transactions on software engineering*, vol. 41, no. 5, pp. 507–525, 2014.

[7] F. Pastore, L. Mariani, and G. Fraser, "Crowdoracles: Can the crowd solve the oracle problem?," in *2013 IEEE Sixth International Conference on Software Testing, Verification and Validation*, pp. 342–351, IEEE, 2013.

[8] T. Y. Chen, F.-C. Kuo, H. Liu, P.-L. Poon, D. Towey, T. Tse, and Z. Q. Zhou, "Metamorphic testing: A review of challenges and opportunities," *ACM Computing Surveys (CSUR)*, vol. 51, no. 1, pp. 1–27, 2018.

[9] B. R. Hoard, "The acceptance, use, and perceptions of metamorphic testing for a sample of open-source software developers," *Cogent Engineering*, vol. 12, no. 1, p. 2522652, 2025.

[10] C. Xu, V. Terragni, H. Zhu, J. Wu, and S.-C. Cheung, "Mr-scout: Automated synthesis of metamorphic relations from existing test cases," *ACM Transactions on Software Engineering and Methodology*, vol. 33, no. 6, pp. 1–28, 2024.

[11] H. Liu, F.-C. Kuo, D. Towey, and T. Y. Chen, "How effectively does metamorphic testing alleviate the oracle problem?," *IEEE Transactions on Software Engineering*, vol. 40, no. 1, pp. 4–22, 2013.

[12] M. Jiang, T. Y. Chen, F.-C. Kuo, D. Towey, and Z. Ding, "A metamorphic testing approach for supporting program repair without the need for a test oracle," *Journal of systems and software*, vol. 126, pp. 127–140, 2017.

[13] H. Jin, Y. Jiang, N. Liu, C. Xu, X. Ma, and J. Lu, "Concolic metamorphic debugging," in *2015 IEEE 39th Annual Computer Software and Applications Conference*, vol. 2, pp. 232–241, IEEE, 2015.

[14] N. Alzahrani, M. Spichkova, and J. Harland, "Application of property-based testing tools\\for metamorphic testing," *arXiv preprint arXiv:2211.12003*, 2022.

[15] T. Y. Chen, S. C. Cheung, and S. M. Yiu, "Metamorphic testing: a new approach for generating next test cases," *arXiv preprint arXiv:2002.12543*, 2020.

[16] S. Segura, G. Fraser, A. B. Sanchez, and A. Ruiz-Cortés, "A survey on metamorphic testing," *IEEE Transactions on software engineering*, vol. 42, no. 9, pp. 805–824, 2016.

[17] F. U. Rehman and M. Srinivasan, "Metamorphic testing for machine learning: Applicability, challenges, and research opportunities," in *2023 IEEE International Conference On Artificial Intelligence Testing (AITest)*, pp. 34–39, IEEE, 2023.

[18] Z.-W. Hui and S. Huang, "Achievements and challenges of metamorphic testing," in *2013 Fourth World Congress on Software Engineering*, pp. 73–77, IEEE, 2013.

[19] T. Y. Chen, F.-C. Kuo, D. Towey, and Z. Q. Zhou, "Metamorphic testing: Applications and integration with other methods: Tutorial synopsis," in *2012 12th International Conference on Quality Software*, pp. 285–288, IEEE, 2012.

[20] T. Y. Chen, J. W. Ho, H. Liu, and X. Xie, "An innovative approach for testing bioinformatics programs using metamorphic testing," *BMC bioinformatics*, vol. 10, no. 1, p. 24, 2009.

[21] F.-C. Kuo, T. Y. Chen, and W. K. Tam, "Testing embedded software by metamorphic testing: A wireless metering system case study," in *2011 IEEE 36th conference on local computer networks*, pp. 291–294, IEEE, 2011.

[22] Z. Q. Zhou, S. Xiang, and T. Y. Chen, "Metamorphic testing for software quality assessment: A study of search engines," *IEEE Transactions on Software Engineering*, vol. 42, no. 3, pp. 264–284, 2015.

[23] W. K. Chan, S. C. Cheung, and K. R. Leung, "A metamorphic testing approach for online testing of service-oriented software applications," *International Journal of Web Services Research (IJWSR)*, vol. 4, no. 2, pp. 61–81, 2007.

[24] S. Segura, R. M. Hierons, D. Benavides, and A. Ruiz-Cortés, "Automated test data generation on the analyses of feature models: A metamorphic testing approach," in *2010 Third International Conference on Software Testing, Verification and Validation*, pp. 35–44, IEEE, 2010.

[25] S. Segura, R. M. Hierons, D. Benavides, and A. Ruiz-Cortés, "Automated metamorphic testing on the analyses of feature models," *Information and Software Technology*, vol. 53, no. 3, pp. 245–258, 2011.

[26] V. Le, M. Afshari, and Z. Su, "Compiler validation via equivalence modulo inputs," *ACM Sigplan Notices*, vol. 49, no. 6, pp. 216–226, 2014.

[27] P. Rao, Z. Zheng, T. Y. Chen, N. Wang, and K. Cai, "Impacts of test suite's class imbalance on spectrum-based fault localization techniques," in *2013 13th International Conference on Quality Software*, pp. 260–267, IEEE, 2013.

[28] X. Xie, W. E. Wong, T. Y. Chen, and B. Xu, "Metamorphic slice: An application in spectrum-based fault localization," *Information and Software Technology*, vol. 55, no. 5, pp. 866–879, 2013.

[29] S. Cho, S. Ruberto, and V. Terragni, "Metamorphic testing of large language models for natural language processing," in *Proceedings of the 41st IEEE International Conference on Software Maintenance and Evolution (ICSME)*, IEEE, 2025.

[30] S. Cho, S. Ruberto, and V. Terragni, "Llmorph: Automated metamorphic testing of large language models," in *2025 40th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, pp. 4102–4105, IEEE, 2025.

[31] Z. Q. Zhou and L. Sun, "Metamorphic testing of driverless cars," *Communications of the ACM*, vol. 62, no. 3, pp. 61–67, 2019.

[32] O. Taipale, K. Smolander, and H. Kälviäinen, "A survey on software testing," *6th International SPICE*, 2006.

[33] N. A. A. Hamid, "Investigating the key factors influencing on management information systems adoption among telecommunication companies in yemen: The conceptual framework development," *International Journal of Energy, Information and Communications*, 2015.

[34] C. Xu, S. Chen, J. Wu, S.-C. Cheung, V. Terragni, H. Zhu, and J. Cao, "Mr-adopt: Automatic deduction of input transformation function for metamorphic testing," in *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*, pp. 557–569, 2024.

[35] M. Vidoni, "A systematic process for mining software repositories: Results from a systematic literature review," *Information and Software Technology*, vol. 144, p. 106791, 2022.

[36] G. Buchgeher, S. Schöberl, V. Geist, B. Dorninger, P. Haindl, and R. Weinreich, "Using architecture decision records in open source projects—an msr study on github," *IEEE Access*, vol. 11, pp. 63725–63740, 2023.

[37] A. A. Khan, J. A. Khan, M. A. Akbar, P. Zhou, and M. Fahmideh, "Insights into software development approaches: mining q &a repositories," *Empirical Software Engineering*, vol. 29, no. 1, p. 8, 2024.